

UỶ BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ

Môn học: Ngôn ngữ lập trình C#

Đề tài: Media Streaming Web Application

GVHD: ThS. Từ Lăng Phiêu

Sinh viên thực hiện:

1. 3124560065 Nguyễn Thành Nhân
2. 3124560022 Lê Khánh Đăng
3. 3124560024 Trần Việt Đức
4. 3124560076 Dương Chí Tài
5. 3124560098 Phan Ngọc Vinh

Thành phố Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

PHẦN 1: PHÂN TÍCH KIẾN TRÚC PHẦN MỀM HỆ THỐNG	1
1.1.Phân tích chi tiết các tầng kiến trúc (Layers)	1
1.2. Lý do TuneVault quyết định lựa chọn Dapper làm ORM chính	2
PHẦN 2: THIẾT KẾ VÀ MÔ TẢ CHI TIẾT 10 CHỨC NĂNG HỆ THỐNG	2
Chức năng 1: Xác thực người dùng (Authentication)	2
Chức năng 2: Hồ sơ người dùng (User Profile).....	2
Chức năng 3: Thư viện Media (Media Library)	3
Chức năng 4: Audio Player (Phát nhạc)	3
Chức năng 5: Video Player (Phát video)	3
Chức năng 6: Quản lý Playlist	4
Chức năng 7: Tìm kiếm & Khám phá	4
Chức năng 8: Chia sẻ Media.....	4
Chức năng 9: Thông báo.....	4
Chức năng 10: Tương tác & Lịch sử	5
PHẦN 3: KHÓ KHĂN KỸ THUẬT VÀ PHƯƠNG ÁN XỬ LÝ	5
Thử thách về tải lên các tệp tin đa phương tiện dung lượng lớn	5
Thử thách về đồng bộ hóa thông báo thời gian thực	6
Thử thách về độ trễ và độ tin cậy của dịch vụ Gemini AI.....	6
Khó khăn trong quản lý truy vấn phức tạp của Dapper	6
KẾT LUẬN	6

PHẦN 1: PHÂN TÍCH KIẾN TRÚC PHẦN MỀM HỆ THỐNG

1.1.Phân tích chi tiết các tầng kiến trúc (Layers)

Kiến trúc Backend của TuneVault bao gồm 4 tầng cốt lõi, phụ thuộc từ ngoài vào trong theo quy tắc Dependency Rule:

- Lớp Domain (TuneVault.Domain): Đây là trung tâm của kiến trúc, chứa các thực thể nghiệp vụ (Entities) như User, MediaItem, Playlist, Artist, Notification, Favorite, Follow, MediaShare. Lớp này định nghĩa các cấu trúc dữ liệu cơ bản và các Interface của Repository (ví dụ: IMediaItemRepository, IUserRepository). Lớp Domain hoàn toàn độc lập, không phụ thuộc vào bất kỳ thư viện hay cơ sở dữ liệu nào.
- Lớp Application (TuneVault.Application): Chứa logic nghiệp vụ ứng dụng, định nghĩa các luồng xử lý (Use Cases). Tầng này triển khai mô hình CQRS (Command Query Responsibility Segregation) thông qua MediatR. Mỗi luồng công việc được phân tách rõ ràng thành Command (ghi dữ liệu) hoặc Query (đọc dữ liệu) kèm theo Handler tương ứng. Ngoài ra, lớp này chứa các Data Transfer Objects (DTOs), các bộ kiểm tra hợp lệ dữ liệu (FluentValidation), và các Service Interface.
- Lớp Infrastructure (TuneVault.Infrastructure): Cung cấp các công nghệ và dịch vụ bên ngoài để phục vụ cho các tầng bên trong. Tầng này chịu trách nhiệm tương tác trực tiếp với SQL Server bằng Dapper (triển khai thông qua lớp DataContextDapper), quản lý tệp tin vật lý trên đĩa cứng (FileStorageService), triển khai dịch vụ thông báo thời gian thực bằng SignalR (NotificationHub), và tích hợp API trí tuệ nhân tạo Gemini AI.
- Lớp API / Presentation (TuneVault.API): Lớp giao tiếp với thế giới bên ngoài. Lớp này chứa các Controller định nghĩa các API Endpoint (RESTful API), xử lý xác thực người dùng bằng JWT Middleware, cấu hình Dependency Injection toàn hệ thống, và quản lý các Middleware xử lý lỗi (ExceptionMiddleware).

Bảng phân tích nhiệm vụ và các thành phần của các lớp trong Clean Architecture

Tầng Lớp (Layer)	Công nghệ / Thư viện chính	Vai trò cốt lõi
TuneVault.Domain	C# Core, No Libraries	Định nghĩa Entities, interfaces Repository
TuneVault.Application	MediatR, FluentValidation	Xử lý Use Case, CQRS Handlers, DTO, Validation
TuneVault.Infrastructure	Dapper, SqlClient, SignalR, HttpClient	Kết nối DB, lưu trữ file, real-time push, API AI Client
TuneVault.API	ASP.NET Core Web API, JWT	Định nghĩa endpoint, Middleware, Authorization

1.2. Lý do TuneVault quyết định lựa chọn Dapper làm ORM chính

Lý do lựa chọn Dapper cho dự án TuneVault hoàn toàn dựa trên các yêu cầu phi chức năng cụ thể của một ứng dụng Streaming:

1. Hiệu năng đọc dữ liệu vượt trội: Ứng dụng streaming yêu cầu tốc độ truy xuất thông tin bài hát, video và lịch sử nghe cực kỳ nhanh chóng. Việc loại bỏ hoàn toàn cơ chế Change Tracking của EF Core giúp Dapper đạt tốc độ truy xuất tối ưu, giảm thiểu độ trễ phản hồi API.
2. Khả năng viết SQL Join tùy biến cao: Trong TuneVault, khi thực hiện lấy thông tin chi tiết một MediaItem, chúng ta cần Join nhiều bảng như Artist, Album,AspNetUsers. Trong Dapper, lập trình viên có thể viết trực tiếp câu lệnh SELECT và ánh xạ các trường như ArtistName, ArtistImage và AlbumName trực tiếp vào MediaItem entity (như đã thiết kế trong ERD). Điều này giúp tránh hiện tượng 'N+1 Query' thường gặp ở EF Core.
3. Tự chủ trong tối ưu hóa SQL: Lập trình viên có thể chủ động viết các câu lệnh chèn dữ liệu nâng cao như 'INSERT INTO ... OUTPUT INSERTED.ID' để lấy trực tiếp khóa chính tự tăng của SQL Server trong một phiên làm việc, tối giản hóa số lượng Roundtrip đến database.

PHẦN 2: THIẾT KẾ VÀ MÔ TẢ CHI TIẾT 10 CHỨC NĂNG HỆ THỐNG

Dưới đây là phân mô tả chi tiết về cách thiết kế, luồng xử lý dữ liệu và cách hiện thực hóa mã nguồn của 10 chức năng tiêu biểu trong hệ thống TuneVault.

Chức năng 1: Xác thực người dùng (Authentication)

- Mô tả: Người dùng có thể đăng ký tài khoản mới, đăng nhập vào hệ thống, làm mới token (tùy chọn) và đăng xuất. Hệ thống phát hành mã JWT Token chứa thông tin định danh (User ID, Email, Role) để Frontend gửi kèm trong các request tiếp theo.
- Luồng xử lý kỹ thuật: Client gửi email và mật khẩu qua HTTP POST. Hệ thống định tuyến qua RegisterCommand hoặc LoginCommand. Handler gọi UserRepository.GetUserByEmailAsync() sử dụng Dapper để truy vấn người dùng từ bảng AspNetUsers. Khi nhận được bản ghi, handler dùng BCrypt.Net để xác thực mật khẩu. Nếu hợp lệ, hệ thống tạo JWT Token bằng JwtSecurityTokenHandler chứa các Claims và trả về LoginResponseDto. Pipeline tương ứng: RegisterCommand, LoginCommand, JWT issuance.

Chức năng 2: Hồ sơ người dùng (User Profile)

- Mô tả: Người dùng có thể xem và chỉnh sửa thông tin hồ sơ cá nhân bao gồm tên hiển thị, ảnh đại diện (avatar) và tiểu sử (bio). Giao diện hiển thị đầy đủ thông tin cá nhân của người dùng.
- Luồng xử lý kỹ thuật: Để xem hồ sơ, Client gửi HTTP GET tới endpoint /api/profile/{id}. API Controller gọi UserRepository để truy vấn thông tin người dùng

bằng Dapper và trả về UserProfileDto. Để cập nhật hồ sơ, Client gửi HTTP PUT mang UpdateProfileCommand bao gồm tên, bio, và file ảnh avatar mới (Multipart Form-data). Handler kiểm tra quyền sở hữu hồ sơ, gọi FileStorageService để lưu ảnh avatar mới nếu có, sau đó thực thi câu lệnh UPDATE trên bảngAspNetUsers qua Dapper. Pipeline tương ứng: UpdateProfileCommand, GET profile.

Chức năng 3: Thư viện Media (Media Library)

- Mô tả: Nghệ sĩ hoặc người dùng có thể tải lên các file âm thanh (.mp3, .wav) hoặc file video (.mp4, .webm) kèm theo metadata như tiêu đề, mô tả và ảnh thumbnail. Hệ thống hỗ trợ phân loại bài hát theo thể loại và album.
- Luồng xử lý kỹ thuật: HTTP POST request mang định dạng Multipart Form-data chứa file và metadata được API nhận qua UploadMediaCommand. UploadMediaValidator kiểm tra định dạng và kích thước tệp (tối đa 20MB cho audio, 100MB cho video). UploadMediaHandler gọi FileStorageService để ghi file vật lý lên đĩa cứng bằng cơ chế streaming trực tiếp và trả về đường dẫn. Sau đó, handler tạo đối tượng MediaItem và gọi MediaItemRepository.CreateMediaAsync() để thực thi câu lệnh INSERT bằng Dapper. Pipeline tương ứng: UploadMediaCommand, streaming endpoint.

Chức năng 4: Audio Player (Phát nhạc)

- Mô tả: Cho phép người dùng phát nhạc trực tuyến mượt mà với đầy đủ chức năng pause, seek và quản lý hàng đợi phát (queue) đơn giản. Giao diện hiển thị thông tin bài đang phát bao gồm tên bài, nghệ sĩ và tiến trình phát.
- Luồng xử lý kỹ thuật: Khi người dùng chọn bài hát, Frontend gửi HTTP GET tới endpoint /api/media/{id}/stream. API Controller truy vấn filePath từ cơ sở dữ liệu qua repository, sau đó dùng FileStreamResult của ASP.NET Core để trả về luồng dữ liệu hỗ trợ Range Request (HTTP 206 Partial Content). Trình duyệt sử dụng thẻ HTML <audio> để phát trực tuyến. Khi bài hát phát được trên 10 giây, Frontend tự động gửi yêu cầu ghi nhận vào RecordPlayHistory. Frontend player bar hiển thị trạng thái phát và quản lý queue ở phía client. Pipeline tương ứng: GET /media/{id}/stream, RecordPlayHistory khi bắt đầu phát.

Chức năng 5: Video Player (Phát video)

- Mô tả: Cho phép người dùng xem video trực tuyến ở chế độ toàn trang (full-page) hoặc trong cửa sổ modal. Hỗ trợ hiển thị ảnh poster thumbnail trước khi video tải và phát trơn tru nhờ kỹ thuật HTTP range request.
- Luồng xử lý kỹ thuật: Frontend sử dụng thẻ HTML <video> với thuộc tính src trỏ tới endpoint /api/media/{id}/stream. Phía Backend, API Controller đọc filePath từ database qua Dapper, kiểm tra MIME type của video (video/mp4, video/webm) và phản hồi bằng FileStreamResult hỗ trợ Range Request (HTTP 206). Kỹ thuật này cho phép người dùng tua đến bất kỳ thời điểm nào trong video mà không cần tải toàn bộ tệp. Ảnh thumbnail được lấy từ trường ThumbnailPath của MediaItem để hiển thị làm poster. Pipeline tương ứng: GET video stream / range requests (khuyến khích).

Chức năng 6: Quản lý Playlist

- Mô tả: Người dùng có thể tạo danh sách phát mới, thêm hoặc xóa bài hát khỏi playlist, đặt chế độ công khai (Public) hoặc riêng tư (Private). Hệ thống hỗ trợ toàn bộ vòng đời CRUD của playlist.
- Luồng xử lý kỹ thuật: Sử dụng MediatR để quản lý các lệnh `CreatePlaylistCommand`, `AddTrackToPlaylistCommand`, `RemoveTrackFromPlaylistCommand`. `CreatePlaylistCommand` nhận tên playlist, mô tả và trạng thái công khai/riêng tư, sau đó thực thi INSERT vào bảng `Playlist`. `AddTrackToPlaylistCommand` kiểm tra quyền sở hữu playlist và tính tồn tại của bài hát, rồi thực thi INSERT vào bảng trung gian `PlaylistTrack`. `RemoveTrackFromPlaylistCommand` xóa bản ghi tương ứng. Tất cả đều tương tác trực tiếp với database qua `Dapper`. Pipeline tương ứng: `CreatePlaylist`, `AddTrackToPlaylist`.

Chức năng 7: Tìm kiếm & Khám phá

- Mô tả: Người dùng có thể tìm kiếm theo tên bài hát, tên nghệ sĩ hoặc tên playlist. Hệ thống hỗ trợ gợi ý nội dung đang thịnh hành (trending) dưới dạng đơn giản và phân trang kết quả tìm kiếm.
- Luồng xử lý kỹ thuật: Client gửi HTTP GET kèm tham số từ khóa tìm kiếm tới endpoint `/api/search`. `SearchMediaQuery` được xử lý bởi handler sử dụng `Dapper` để thực thi câu lệnh SQL với mệnh đề LIKE trên các cột `Title`, `ArtistName`. Kết quả được phân trang với tham số `PageNumber` và `PageSize`. Để lấy trending, hệ thống truy vấn bảng `PlayHistory` để đếm số lượt nghe trong 7 ngày gần nhất, sắp xếp giảm dần và trả về danh sách `MediaItem` phổ biến nhất. Pipeline tương ứng: `SearchMediaQuery` với phân trang.

Chức năng 8: Chia sẻ Media

- Mô tả: Người dùng có thể gửi chia sẻ một bài hát hoặc toàn bộ playlist cho người dùng khác trong hệ thống. Người nhận có thể xem danh sách các nội dung đã được chia sẻ đến mình trong mục “Đã chia sẻ với tôi”.
- Luồng xử lý kỹ thuật: Client gửi HTTP POST mang `ShareMediaCommand` bao gồm `ReceiverID` và `MediaItemID` (hoặc `PlaylistID`). Handler kiểm tra tính hợp lệ của `ReceiverID` và đối tượng được chia sẻ bằng `Dapper`. Sau đó gọi `ShareRepository.CreateShareMediaAsync()` để ghi bản ghi vào bảng `MediaShare`. Đồng thời, một bản ghi thông báo mới được chèn vào bảng `Notification` với trạng thái `IsRead = 0`, và hệ thống gọi `SignalR Notification Hub` để đẩy thông báo thời gian thực đến người nhận. Pipeline tương ứng: `ShareMediaCommand`, bảng `MediaShare`.

Chức năng 9: Thông báo

- Mô tả: Khi người dùng được chia sẻ nhạc, được người khác follow hoặc playlist của họ được chia sẻ, một thông báo lập tức xuất hiện trên màn hình mà không cần tải lại trang. Người dùng có thể đánh dấu thông báo là đã đọc.

- Luồng xử lý kỹ thuật: Áp dụng quy trình Store-and-Forward. Khi sự kiện xảy ra, hệ thống luôn ghi bản ghi vào bảng Notification với IsRead = 0 trước. Lớp NotificationHub kế thừa Hub của SignalR được đăng ký tại API layer. Các Handler tiêu chuẩn INotificationPushService và gọi SendNotificationAsync(userID, notification). Dịch vụ này dùng IHubContext<NotificationHub> để định vị kết nối WebSocket của người dùng và gọi phương thức ReceiveNotification trên client. Khi Frontend khởi chạy, nó gửi GET tới API để tải danh sách thông báo chưa đọc. Pipeline tương ứng: SignalR Hub, Notification entity, mark as read.

Chức năng 10: Tương tác & Lịch sử

- Mô tả: Người dùng có thể đánh dấu yêu thích (like/favorite) một bài hát hoặc video để lưu vào thư viện cá nhân. Hệ thống tự động ghi lại lịch sử nghe nhạc gần đây, hiển thị tối đa 10 bài mới nhất.

- Luồng xử lý kỹ thuật: ToggleFavoriteCommand gửi cặp khóa (UserID, MediaItemID). Handler kiểm tra bản ghi trong bảng Favorite qua Dapper. Nếu chưa tồn tại, thực hiện INSERT; nếu đã tồn tại, thực hiện DELETE (toggle). Để ghi lịch sử nghe, khi bài hát phát trên 10 giây, Frontend gửi RecordPlayHistoryCommand và handler thực thi INSERT vào bảng PlayHistory với thời gian UTC hiện tại. Khi truy vấn lịch sử gần đây, hệ thống SELECT TOP 10 từ bảng PlayHistory ORDER BY PlayedAt DESC. Dữ liệu lịch sử nghe cũng là đầu vào cho thuật toán gợi ý AI. Pipeline tương ứng: ToggleFavoriteCommand, RecordPlayHistoryCommand.

PHẦN 3: KHÓ KHĂN KỸ THUẬT VÀ PHƯƠNG ÁN XỬ LÝ

Trong quá trình thiết kế và phát triển TuneVault, nhóm đã đối mặt với một số thử thách kỹ thuật quan trọng. Dưới đây là mô tả chi tiết các khó khăn và phương án kỹ thuật đã được áp dụng để giải quyết triệt để.

Thử thách về tải lên các tệp tin đa phương tiện dung lượng lớn

- Khó khăn: Theo mặc định, máy chủ ASP.NET Core (Kestrel) giới hạn dung lượng tối đa của một request body là khoảng 30MB. Khi người dùng tải lên các video chất lượng cao (.mp4) có dung lượng lớn hơn 100MB, máy chủ sẽ lập tức từ chối kết nối và trả về lỗi HTTP 413 Payload Too Large. Ngoài ra, việc đọc toàn bộ tệp vào bộ nhớ đệm (RAM) trước khi ghi xuống ổ đĩa có thể gây ra hiện tượng tràn bộ nhớ máy chủ (Out of Memory) khi có nhiều người dùng cùng tải lên đồng thời.

- Giải pháp xử lý: nhóm đã cấu hình lại giới hạn kích thước request trong file Program.cs bằng cách điều chỉnh cấu hình Kestrel và FormOptions. Đồng thời, hệ thống sử dụng cơ chế streaming file trực tiếp (Direct File Streaming) bằng cách mở luồng FileStream trực tiếp từ tệp tin gửi lên và ghi tuần tự vào đĩa cứng từng phân đoạn dữ liệu (chunks), thay vì tải toàn bộ tệp tin vào RAM.

Thử thách về đồng bộ hóa thông báo thời gian thực

- **Khó khăn:** Sử dụng kết nối SignalR (WebSocket) để đẩy thông báo yêu cầu người dùng phải đang trực tuyến (Online). Nếu người dùng A chia sẻ một bài hát cho người dùng B khi người dùng B đang ngoại tuyến (Offline), kết nối SignalR của người dùng B sẽ không khả dụng. Thông báo này sẽ bị mất nếu hệ thống chỉ truyền tin real-time đơn thuần mà không có cơ chế lưu trữ.
- **Giải pháp xử lý:** Áp dụng quy trình lưu trữ trước - phát sóng sau (Store-and-Forward). Khi sự kiện chia sẻ hoặc follow xảy ra, hệ thống luôn ghi một bản ghi vào bảng Notification trong cơ sở dữ liệu trước với trạng thái chưa đọc (IsRead = 0). Sau đó, hệ thống mới kiểm tra kết nối SignalR. Nếu người dùng trực tuyến, thông báo sẽ được đẩy trực tiếp. Đồng thời, khi Frontend của người dùng khởi chạy hoặc phát hiện kết nối lại mạng, nó sẽ tự động gửi yêu cầu GET tới API Controller để tải về danh sách các thông báo chưa đọc từ cơ sở dữ liệu.

Thử thách về độ trễ và độ tin cậy của dịch vụ Gemini AI

- **Khó khăn:** Việc gọi API bên ngoài (Gemini AI API) thông qua Internet thường phát sinh độ trễ lớn (có thể mất từ 2 đến 5 giây để nhận kết quả phản hồi). Ngoài ra, API này có thể gặp lỗi kết nối hoặc bị giới hạn số lượt gọi (Rate-limit) khiến nghiệp vụ tải trang đề xuất âm nhạc của người dùng bị treo hoặc bị lỗi.
- **Giải pháp xử lý:** Áp dụng cơ chế Fallback thuật toán nội bộ kết hợp với xử lý ngoại lệ (Exception Handling). Trong GetRecommendationsQueryHandler, toàn bộ logic gọi Gemini AI được bọc trong một khối try-catch. Nếu API AI phản hồi thành công, kết quả sẽ được sử dụng. Trong trường hợp API gặp lỗi hoặc quá thời gian phản hồi (Timeout), hệ thống sẽ tự động kích hoạt thuật toán gợi ý thay thế: tìm kiếm ngẫu nhiên 5 bài hát của nghệ sĩ được người dùng nghe nhiều nhất trong lịch sử nghe nhạc gần đây để làm kết quả đề xuất. Điều này đảm bảo tính năng gợi ý luôn hoạt động ổn định 100% đối với người dùng.

Khó khăn trong quản lý truy vấn phức tạp của Dapper

- **Khó khăn:** Vì Dapper không tự động quản lý mối quan hệ giữa các thực thể, khi cần lấy thông tin bài hát cùng với tên của Nghệ sĩ (Artist) và tên của Album, lập trình viên phải tự viết các lệnh JOIN phức tạp. Nếu cấu trúc bảng thay đổi (ví dụ: thêm trường Bio vào bảng Artist hay AspNetUsers), các chuỗi SQL thô trong code sẽ bị lỗi thời và khó phát hiện ở thời điểm biên dịch (Compile-time).
- **Giải pháp xử lý:** Chuẩn hóa toàn bộ câu lệnh SQL trong lớp Repository, tập hợp các truy vấn vào một nơi duy nhất. Sử dụng kỹ thuật Multi-mapping của Dapper để ánh xạ các đối tượng liên quan một cách an toàn và tự động cập nhật DTO tương ứng. Điều này giúp giảm thiểu lỗi cú pháp SQL và đảm bảo tính đồng bộ dữ liệu cao.

KẾT LUẬN

Báo cáo trên đã phân tích chi tiết toàn bộ thiết kế hệ thống, kiến trúc phân tầng Clean Architecture, lý do lựa chọn thư viện tương tác dữ liệu tối giản Dapper để tối ưu hóa

hiệu năng, mô tả 10 chức năng cốt lõi và các giải pháp vượt qua các thử thách kỹ thuật quan trọng của dự án TuneVault.

Hệ thống TuneVault đã hiện thực hóa thành công một mô hình ứng dụng Web Streaming hiện đại, có tính mở rộng cao nhờ áp dụng mẫu thiết kế CQRS cùng MediatR, tối ưu hóa tốc độ cơ sở dữ liệu bằng Dapper, tăng cường khả năng tương tác real-time bằng SignalR và áp dụng trí tuệ nhân tạo Gemini AI để tối ưu hóa trải nghiệm cá nhân hóa của người dùng. Đây là một nền tảng vững chắc để tiếp tục phát triển thêm các tính năng cao cấp hơn trong tương lai.